浙江大学



设计题目　老年人随心手套

课程名称　数字系统

指导老师　李宇波　徐楚佳

姓名学号　郑智瑄　3230103913

李明远　3230101544

赵欧凯　3200104954

学　　院　信息与电子工程学院

2024 年 06 月 17 日

目录

一， 设计理念 3
二， 项目展示 4
2.1 整体连接展示图 4
2.2 VGA 显示示意图 4
2.3 电子锁展示图 5
2.4 蓝牙点灯展示图 6
2.5 温度显示展示图 6
三．模块功能 7
3.1 整体模块展示 7
3.2 传感器模块 8
3.3 过渡模块 10
3.4 VGA 模块 12
3.5 音乐播放器模块 17
3.6 温度显示模块 20
3.7 电子锁模块 23
3.8 通讯模块 24
四．心得体会 27
4.1 郑智瑄： 27
4.2 李明远： 27
4.3 赵欧凯： 28
五．成员分工 30
5.1 进度介绍： 30
5.2 分工与贡献比; 30
六、 物资清单与价格 31

一， 设计理念

随着科技的不断进步，特别是计算机软硬件和互联网技术的高速发展，人们期望的万物互联的构想正在一步步成为现实，各种各样的智能家居已经极大的方便了人们的日常生活。与此同时，诸如智能手环、智能眼镜等可穿戴式智能设备的形态开始多样化，逐渐在工业、医疗健康、军事、教育、娱乐等领域表现出广阔的应用潜力。特别的，随着全球人口老龄化的加剧，智慧养老成为了一个越来越重要的议题，而可穿戴设备在智慧养老领域的应用背景是一个充满潜力和挑战的领域。

本设计旨在利用可穿戴智能设备便携性、实时性和个性化的特点，为老年人提供更加安全、舒适和便利的生活环境。我们期望通过 Ego1FPGA 开发板实现类似于智能遥控的功能，模拟控制各种智能家居。我们的设计在两块开发板上实现，第一块板子起到一个控制器的作用，在接收到操作信号后，其将信号传输到第二块开发板上来实现显示。我们的设计更偏向于一个可搭载平台，可以通过在第二块开发板上增删模块来实现多功能设计的目的。

鉴于自身能力的局限性、模块本身大小和模块之间连接的限制，我们无法使设计具有理想手环那样足够的便携性，在设计性能上也存在一些不足。但我们已经竭尽全力的实现了我们所能完成的部分，这将是我们在专业设计和未来学习路上的坚实一步。

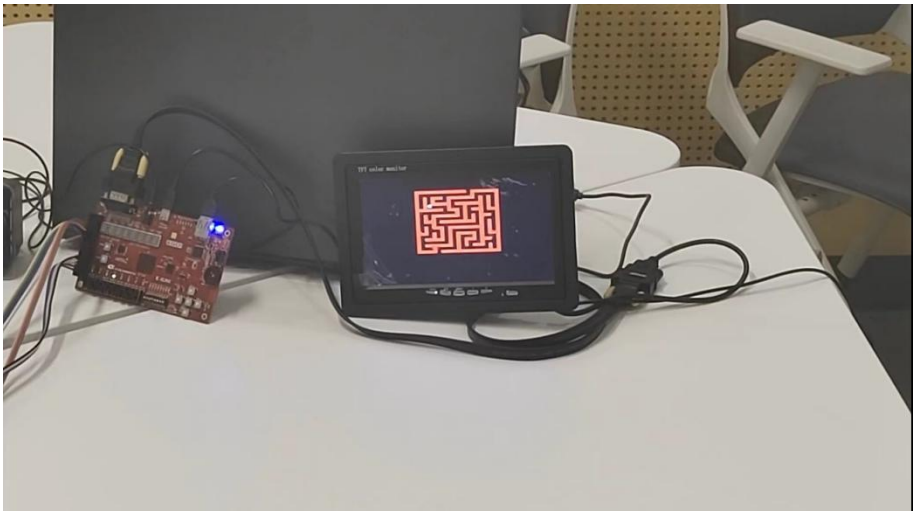
二， 项目展示

2.1 整体连接展示图

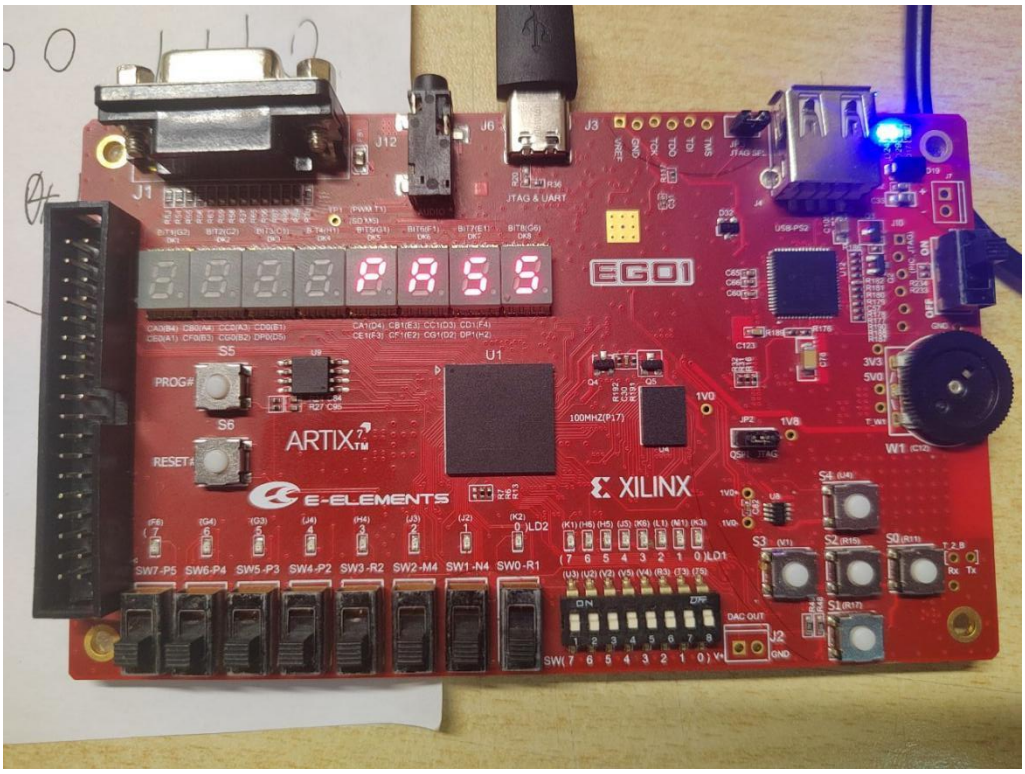


2.2 VGA 显示示意图

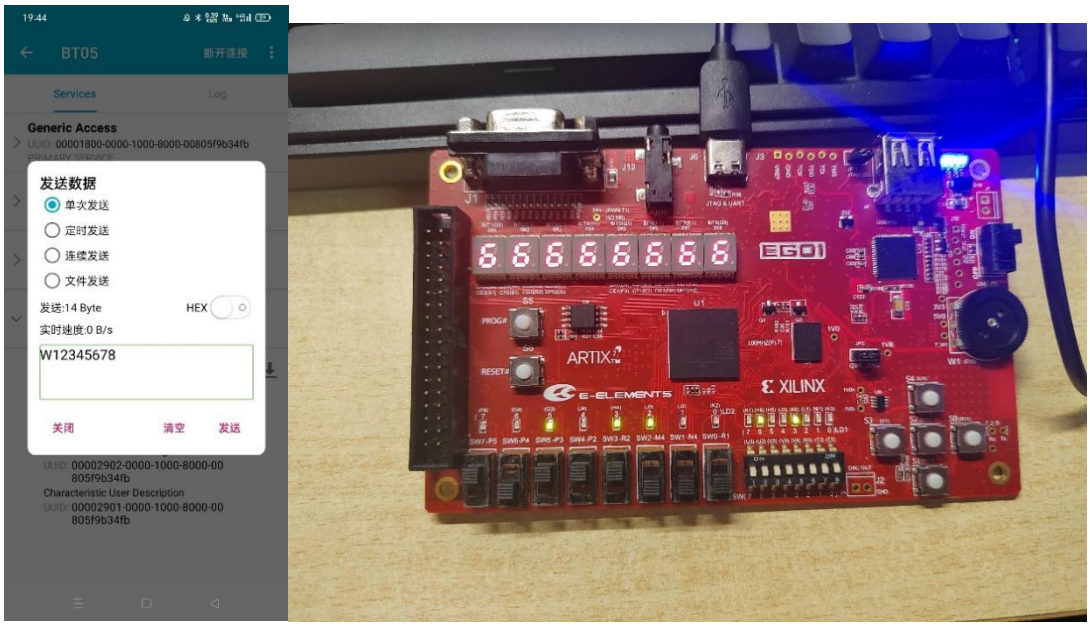




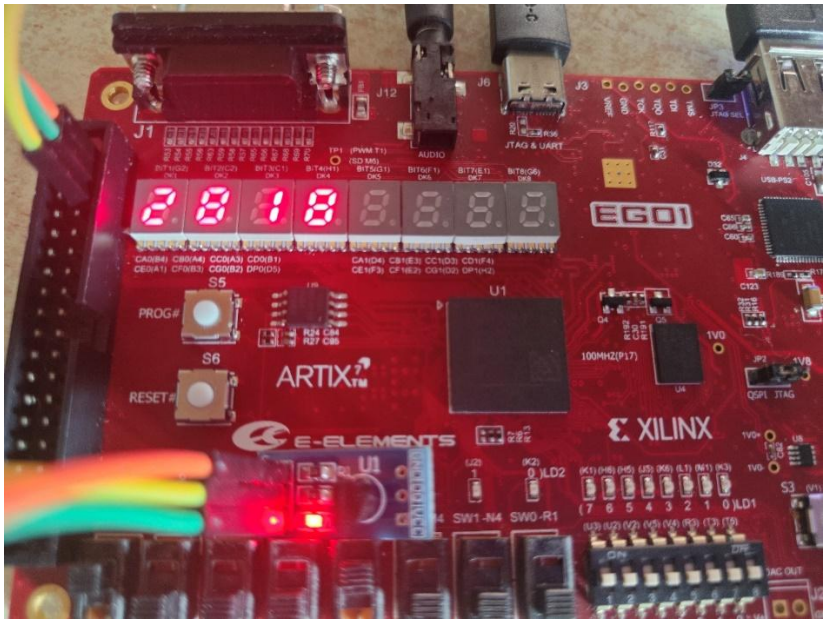
2.3 电子锁展示图


P A S S
EGO1
ARTIX7
XILINX
E-ELEMENTS
PROG#
RESET#
JTAG & UART
100MHZ(P17)
SW7-P5 SW6-P4 SW5-P3 SW4-P2 SW3-R2 SW2-M4 SW1-N4 SW0-R1

2.4 蓝牙点灯展示图



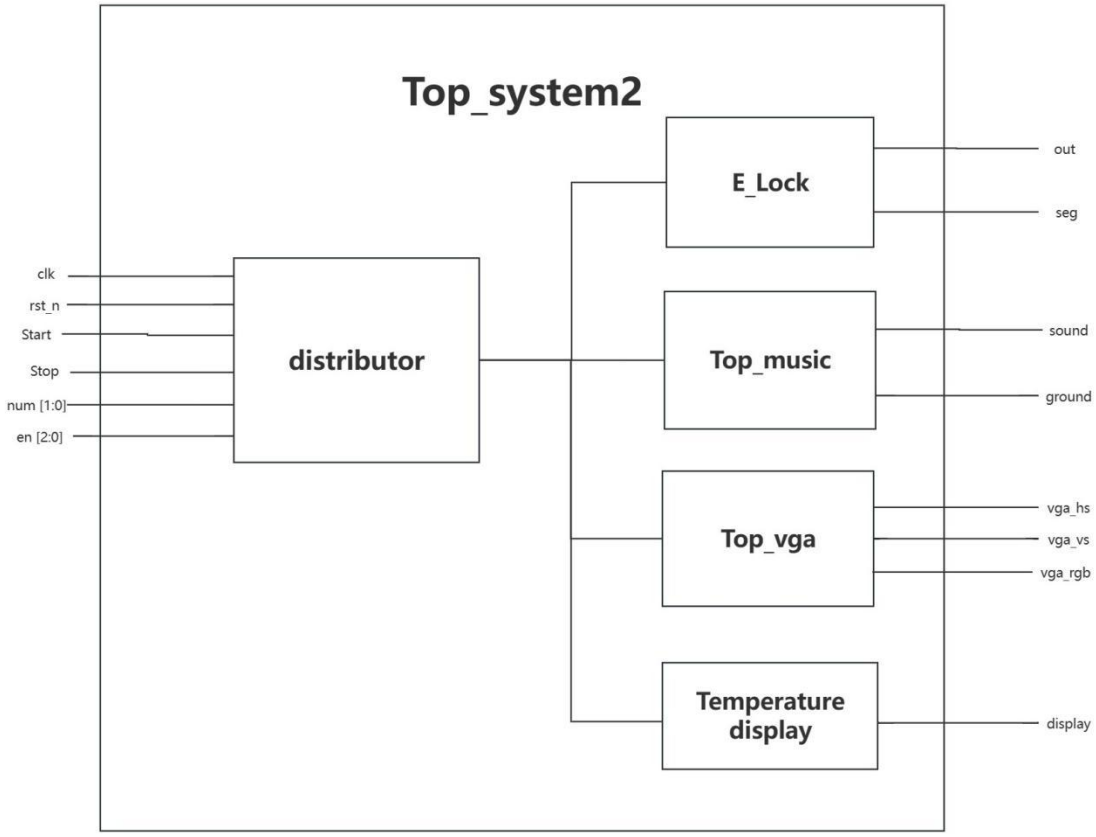
2.5 温度显示展示图



三．模块功能

3.1　整体模块展示

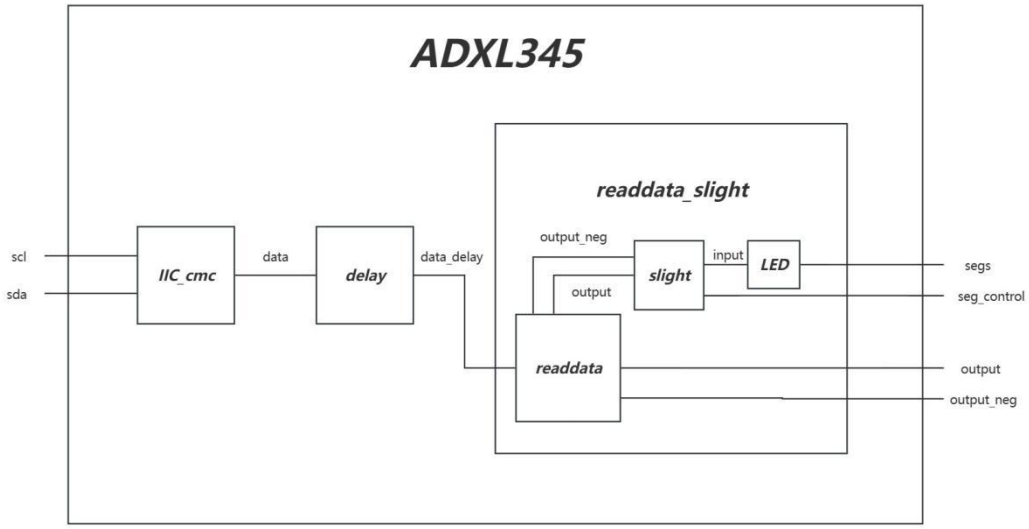




我们的设计主要可以分为传感器模块，过渡模块，与接受总模块，其中接受总模块包含 VGA 模块，音乐播放器等多个模块。下面将分别介绍各模块的功能和功能的实现。

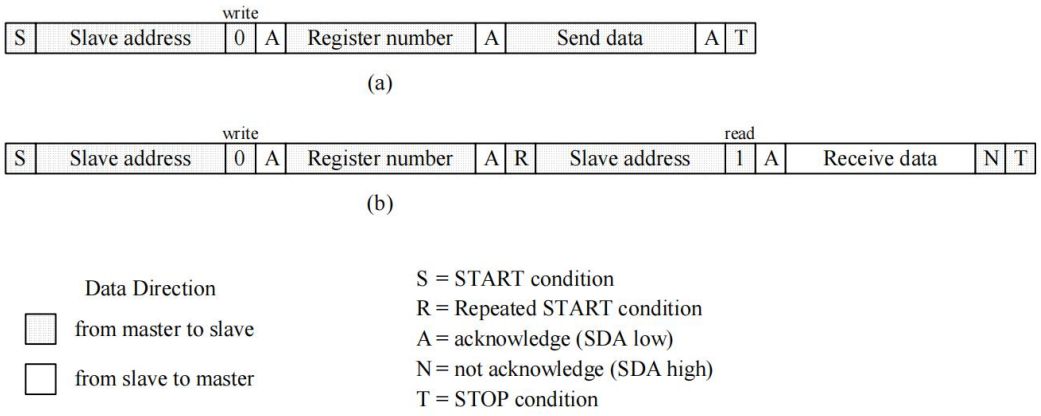
3.2 传感器模块

我们使用 ADXL345 加速度传感器来设计传感器模块。

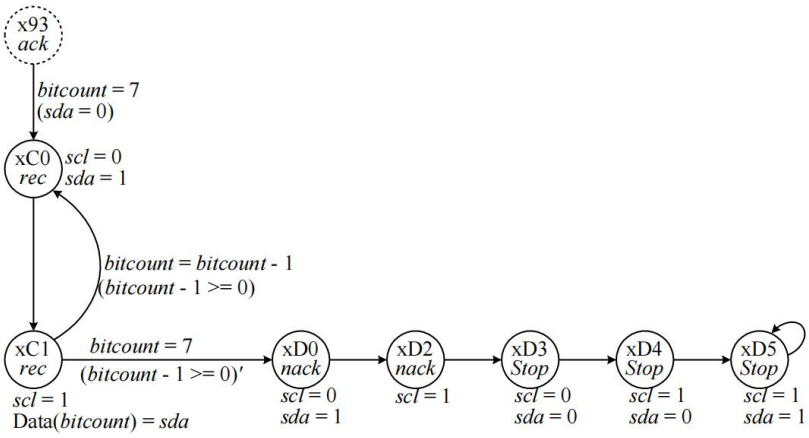


该系统主要由以下几个部分组成：

IIC_cmc：这是 I²C 通信接口模块，用于与外部设备（如 ADXL345）进行数据交换。



（IIC 时序图）



(b)

图 9.46 I²C 的主体控制器的部分状态图 (a)发送开始条件和从体地址 (b) 接收从体发来的数据和发送停止条件

（时序图的状态图）

delay：这是一个延迟模块，用于处理 I²C 通信中加速度传感器传入数据速度过快。

readdata：这是一个读取数据并判别是否达到阈值的模块，它接收来自 delay 的数据，并生成输出到 LED 和其他控制信号。

readdata_slight：一个读取数据并判别是否达到阈值的模块，其功能类似于 readdata，它接收来自 delay 的数据，并生成输出到七段数码管和其他控制信号。

该系统的功能：

readdata 和 readdata_slight。这两个模块都接受一个 17 位的数据输入（data_delay），并从中提取出整数部分（Oint）和小数部分（Ofrac）。然后，这些值被用来驱动 LED 灯或其他显示设备。readdata 模块将整数和小数部分转换为二进制表示，并通过逻辑运算来确定 LED 的状态。如果整数部分是 1 或小数部分超过约 0.8，则 LED 亮起；否则，LED 熄灭。

readdata_slight 模块比 readdata 多了一个名为 slight 的子模块。slight 模块是一个数字显示器控制器，它将整数、小数和额外的常数值（O4：g）转换为 4 位 BCD 码，并将其发送到 LED 阵列（Segs）以显示出来。

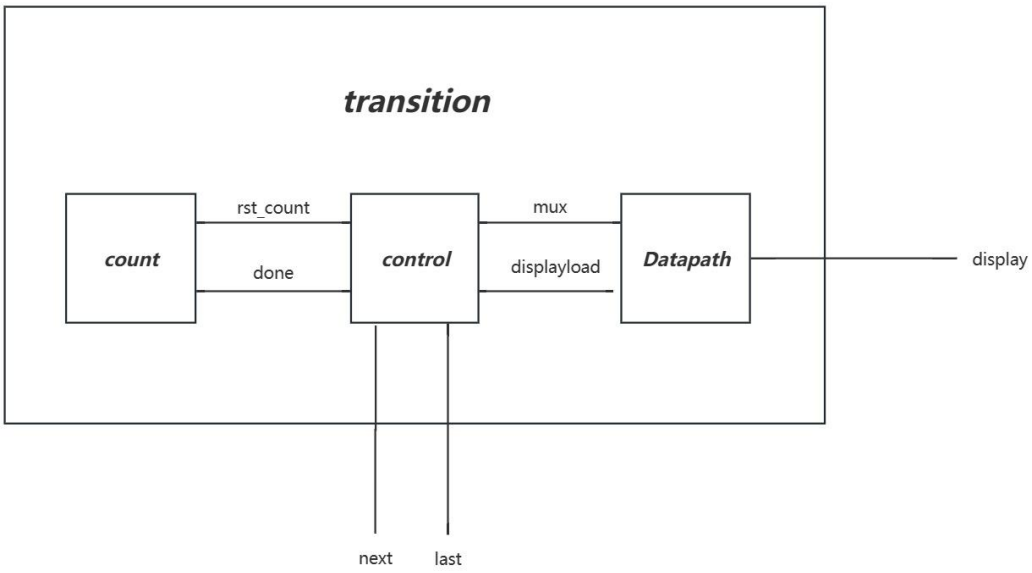
这个系统的主要功能是从 ADXL345 获取数据，并通过 LED 灯与七段数码管显示出来。具体来说，它会将加速度数据转换成易于理解的形式，并用 LED 灯与七

段数码管来表示。

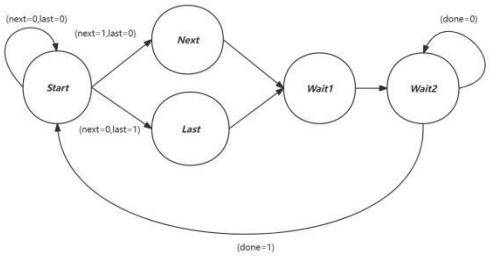
核心代码展示

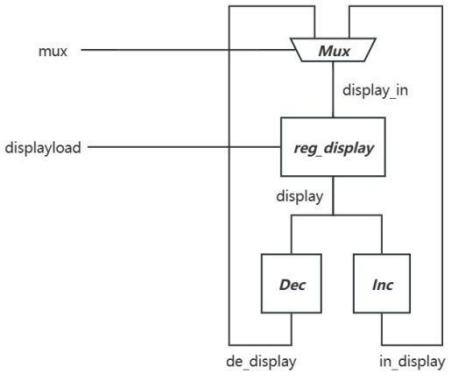
```
module readdata_slight(
    input [16:0] data_delay,
    input sys_clk,rst,
    output LED,
    output LED_neg,
    output [3:0]  seg_control,
    output [7:0]  Segs
);
wire [3:0] 01,02,03,04,0,02_dot;
wire [3:0] 0int,0frac;
assign 0int=8*data_delay[11]+4*data_delay[10]+2*data_delay[9]+data_delay[8];
assign 0frac=(50000*data_delay[7]+25000*data_delay[6]+12500*data_delay[5]+6250*data_delay[4]+3125*data_delay[4])/10000;
assign 01[3]=data_delay[16];
assign 01[2]=data_delay[16];
assign 01[1]=data_delay[16];
assign 02=0int%10;
assign 02_dot=02+10;
assign 03=0frac;
assign 04=15;
wire  clk_2ms;
assign LED=~01[3]&(02[3]|02[2]|02[1]|02[0]|03[3]);
assign LED_neg=01[3]&(02[3]|02[2]|02[1]|02[0]|03[3]);
clk_div #(.N(10_0000)) c2(.clk(sys_clk),.reset(rst),.clk_div(clk_2ms));
slight #(.N(4)) s2(.01(01),.02(02_dot),.03(03),.04(04),
.clk(clk_2ms),.reset(rst),.seg_control(seg_control),.0(0));
LED L1(.I(0),.Segs(Segs));
endmodule
```

3.3 过渡模块



过渡模块状态图





过 渡 模 块 Datapath

过渡模块控制字

过渡模块控制字			
	mux	displayload	rst_count
Start	X	0	X
Next	0	1	X
Last	1	1	X
Wait1	X	0	1
Wait2	X	0	0

该模块设计用于控制音乐显示相关的状态机逻辑，包括对数值的增减操作以及状态之间的转换。它接收几个输入信号并产生一个输出,同时内部包含状态机、计数器、多路复用器、加法器、减法器和寄存器等组件。

功能描述

状态机:定义了一个五状态的状态机,状态转移逻辑由外部输入 next 和 last 控制，以及内部信号 done 决定是否从状态 4 回到状态 0。

状态 0：初始化，根据 next 和 last 选择下一个状态。

状态 1 至 4：涉及不同的操作，如选择数据源、加载寄存器等，状态 4 等待 done 信号以返回初始状态或保持当前状态。

数据通路：

计数器 (count)：接收 rst_count 作为复位信号和 clk 作为时钟信号，当计数达到预设值（15,000,000）时，输出 done 信号，用于状态机的控制。

多路复用器 (mux2to1)：根据 mux 信号选择 de_display（减法结果）或 in_display（加法结果）作为 display_in。

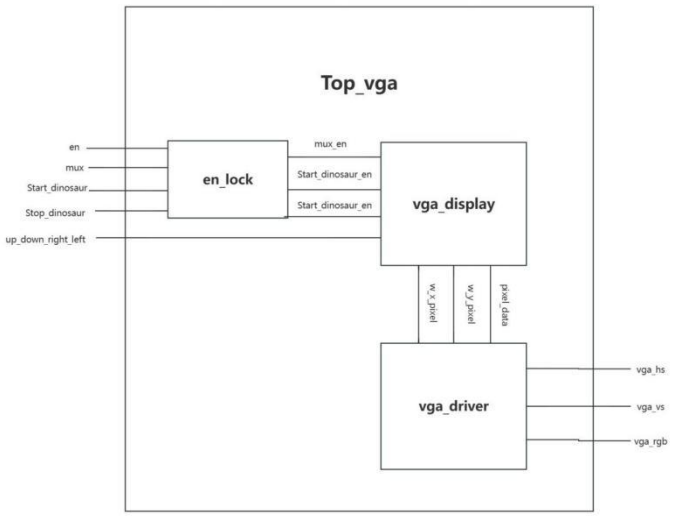
加法器 (Increment) 和 减法器 (Decrement)：分别对 display 的值进行加 1 或减 1 操作，结果输出到 in_display 和 de_display。

寄存器 (Register)：根据 clk、reset、displayload 和 display_in 控制信号，实现数据的加载和清零操作。当 displayload 为 1 时，从 display_in 加载数据；当 reset 为 1 时，寄存器被清零。

总体功能

该模块通过一个状态机控制一系列操作，包括选择是进行加法还是减法操作（通过改变数据流路径），并在特定时刻将计算结果加载到显示寄存器中。外部的 next 和 last 信号允许外部控制状态的推进或回退，整个模块设计用于管理音乐播放或显示相关的控制流程，实现在输入变量第一次改变时“瞬时响应”后续“延时响应”。

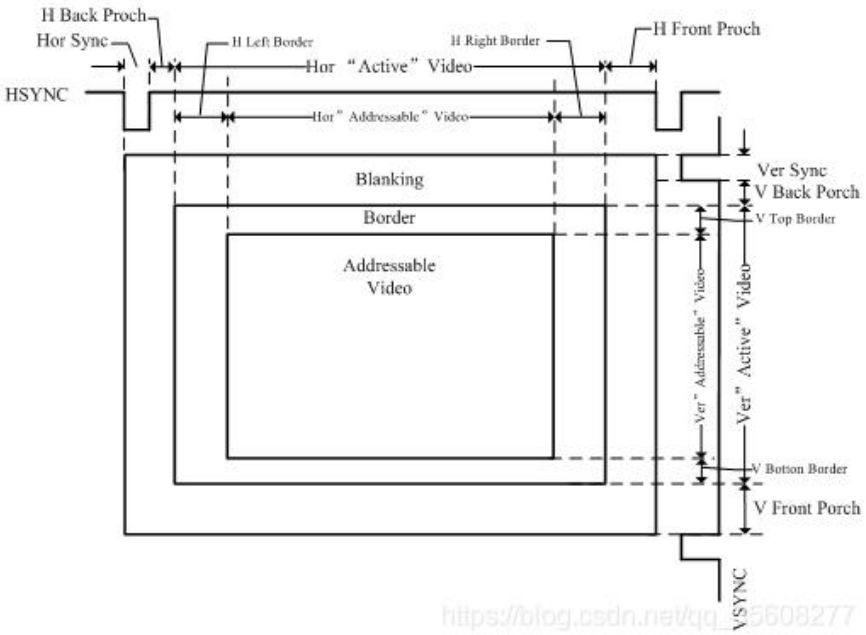
3.4 VGA 模块



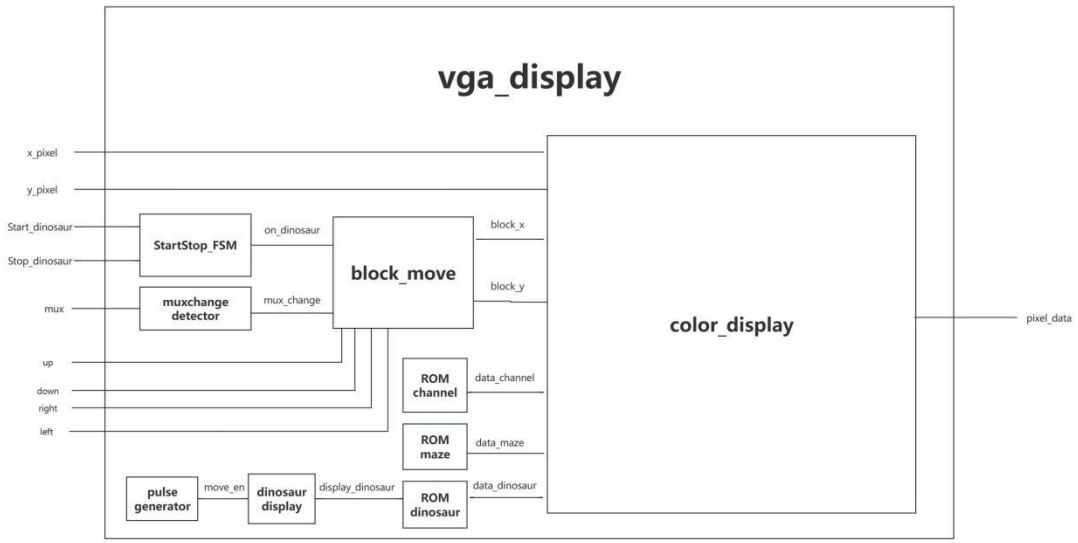
（VGA 模块图）

显示模式	时钟	行时序（像素数）					帧时序（行数）				
	（MHz）	a	b	c	d	e	o	p	q	r	s
640x480@60	25.175	96	48	640	16	800	2	33	480	10	525
640x480@75	31.5	64	120	640	16	840	3	16	480	1	500
800x600@60	40.0	128	88	800	40	1056	4	23	600	1	628
800x600@75	49.5	80	160	800	16	1056	3	21	600	1	625
1024x768@60	65	136	160	1024	24	1344	6	29	768	3	806
1024x768@75	78.8	176	176	1024	16	1312	3	28	768	1	800
1280x1024@60	108.0	112	248	1280	48	1688	3	38	1024	1	1066
1280x800@60	83.46	136	200	1280	64	1680	3	24	800	1	828
1440x900@60	106.47	152	232	1440	80	1904	3	28	900	1	932

（VGA 模式显示图）



（VGA 时序图）



（VGA 核心模块）

该代码段是一个 Verilog 模块定义，命名为 top_vga，它构成了一个顶层模块，旨在控制一个 VGA（视频图形阵列）显示系统。该模块整合了时钟分频、VGA 驱动逻辑、以及 VGA 显示处理逻辑，支持动态切换显示内容（通过 mux 信号）和控制一个恐龙动画的开始与停止（通过 Start_dinosaur 和 Stop_dinosaur）。以下是该模块的主要组成部分与功能的详细说明：

功能模块

clk_div: 时钟分频模块，将输入时钟 clk 按照配置分频，产生适合 VGA 时序要求的工作时钟 w_clk。

vga_driver (B): VGA 驱动模块，接收工作时钟 w_clk、复位信号 rst_n、像素颜色数据 pixel_data，以及像素坐标 w_x_pixel 和 w_y_pixel，输出 VGA 同步信号 vga_hs 和 vga_vs，以及最终的彩色信号 vga_rgb，以驱动 VGA 显示器。

此模块的核心作用是在给定的 VGA 时序约束下，精确地控制像素数据的输出时机，确保视频信号与显示器的刷新频率匹配，从而正确显示图像。它通过精细的时序控制实现了 VGA 信号的生成，包括正确的同步信号插入，以及在有效显示窗口内传输像素数据，是构建任何基于 VGA 输出系统的关键部分，如简单的图形界面、游戏或者教学演示设备等。

vga_display (C): VGA 显示处理模块，根据当前像素坐标、颜色数据以及外部控制信号（包括显示模式选择 mux_en，以及恐龙动画的控制信号 Start_dinosaur_en 和 Stop_dinosaur_en），处理显示逻辑，可能包括绘制静态画面、切换显示模式或控制动画。此外，该模块还能响应用户控制（上、下、左、右移动），实现交互式功能

vga_display 模块包含了多个子模块和功能块。以下是对各个部分的简要解释：

StartStop_FSM: 这个模块是用来控制恐龙动画的开始和停止的有限状态机（FSM），它根据`Start_dinosaur`和`Stop_dinosaur`信号来触发相应的动作。

muxchange detector: 这个模块检测`mux`信号的变化，并将其转换成一个控制信号`mux_change`。

block_move: 这个模块是一个运动控制器，用于控制屏幕上的方块移动，它连接到了`color_display`模块。

ROM channel: 这个模块存储着频道的数据，可以理解为一种背景纹理。

ROM maze：这个模块存储迷宫数据，用于创建迷宫背景或游戏环境。

ROM dinosaur：这个模块存储恐龙的图像数据，用于显示恐龙动画。

color_display:这个模块是核心显示模块，它接收来自其他模块的数据，并将它们组合在一起，然后输出到`pixel_data`，以便在屏幕上显示出来。

dinosaur display:是专门用于显示两种恐龙动画的模块,它可能依赖于`ROM dinosaur`中的数据。

pulse generator 这个模块生成一些定时信号，用于控制恐龙动画移动或者其他需要定时的操作。

move_en：这个信号是一个使能信号，用于控制方块的移动。

block_x/block_y：这些是用于记录方块位置的变量。

data_channel/data_maze:这些是从对应的 ROM 读取出来的数据，用于渲染背景。

总体来说，`vga_display`模块的功能是提供一个互动式的显示界面，其中有三个频道播放恐龙移动和一个可移动的方块走迷宫。用户可以通过 up, down, right, left 信号来控制方块的移动，同时也可以通过 Start_dinosaur 和 Stop_dinosaur 信号来开启或关闭恐龙动画。

总体功能

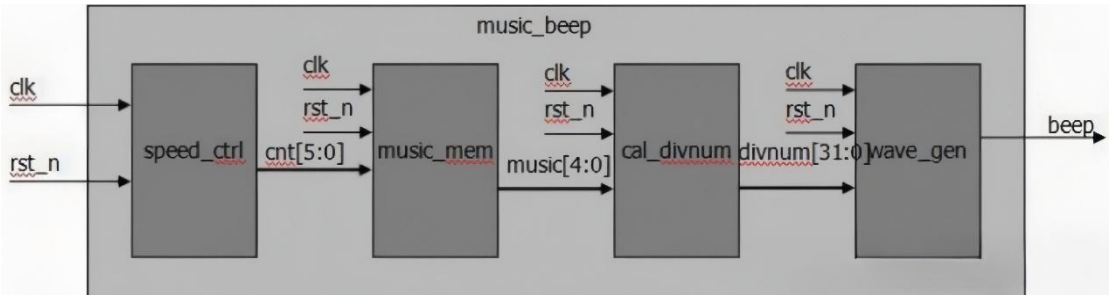
该 top_vga 模块综合了时序控制、VGA 信号生成和显示逻辑处理，形成一个完整的 VGA 显示控制系统。它可以动态地在不同的显示模式间切换（通过 mux 信号），并能够控制一个特定的恐龙动画播放与迷宫游戏，同时支持用户通过方向键进行交互。

VGA 核心代码：

```
//block move
always@(posedge clk_div or posedge rst_n) begin
    if(rst_n)
        begin
            if(mux<3)
            block_x <= begin_x;
            block_y <= begin_y;
        end
    else begin
    if(mux<3) begin
        if(mux_change==1) begin
            block_x <= begin_x;
            block_y <= begin_y;
        end
        else begin
            if(on_dinosaur) begin
                if(move_en)
                    if(block_x == end_x)
                        block_x <= begin_x;
                    else
                        block_x <= block_x + length/2;
            end
        end
    end
    if(mux==3) begin
        if(mux_change==1) begin
            block_x <= begin_xm;
            block_y <= begin_ym;
        end
        else
            if(up==1 && block_y != begin_ym && en_up==1) block_y <= block_y - 1;
            if(down==1 && block_y != end_ym && en_down==1) block_y <= block_y + 1;
            if(right==1 && block_x != end_xm && en_right==1) block_x <= block_x + 1;
            if(left==1 && block_x != begin_ym && en_left==1) block_x <= block_x - 1;
    end
    end
end
```

```
always@(posedge vga_clk or posedge rst_n)
begin
    if(rst_n)
        pixel_data <= black;
    else begin
        if (mux<3)begin
            if(x_pixel >= 220 && x_pixel <= 412 && y_pixel >= 168 && y_pixel <= 232)
                if(char[y_pixel-168][412-x_pixel])
                pixel_data <= white;
                else
                pixel_data <= black;
            else
                if (x_pixel >= block_x  && (block_x + length) > x_pixel && y_pixel >= ( block_y  )  && y_pixel < (block_y + wide) )
                    if(data_dinosaur[y_pixel-block_y][block_x + length-10'b1-x_pixel])
                    pixel_data <= white;
                    else
                    pixel_data <= black;
                else
                pixel_data <= black;
        end
        if(mux==3) begin
            if (x_pixel >= block_x  && (block_x + lengthm) > x_pixel && y_pixel >= ( block_y  )  && y_pixel < (block_y + widem) )
                pixel_data <= white;
            else if(x_pixel >= begin_xmaze && x_pixel <= end_xmaze && y_pixel >= begin_ymaze && y_pixel <= end_ymaze)
                if(maze[y_pixel-begin_ymaze][end_xmaze-x_pixel])
                pixel_data <= red;
                else
                pixel_data <= black;
            else
                pixel_data <= black;
        end
        end
end
endmodule
```

3.5 音乐播放器模块



音乐播放器（music beep）系统的设计分为

时钟信号（clk）：这是数字电路的核心，提供同步信号，使电路中的各个部分能够按照统一的节奏工作。

复位信号（rst_n）：这是一个低电平有效的复位信号，用于将系统重置到初始状态。

蜂鸣器控制（beep, speed_ctrl）：可能表示蜂鸣器的控制信号和速度控制信号，用于调节蜂鸣器的音调或节奏。

计数器（cnt[5:0]）：这是一个6位的计数器，用于控制时钟分频、音乐节奏或音调。

音乐存储器（music_mem）：这可能是一个存储音乐旋律或音符数据的存储器，用于音乐播放。

分频器（cal_divnum, divnum[31:0]）：分频器用于生成不同频率的时钟信号，用于调节音乐节奏或音调。

波形生成器（wave_gen）：用于生成特定波形的信号，与音乐的音调有关。

音乐输出（music[4:0]）：这是一个5位的输出，代表生成的音乐信号

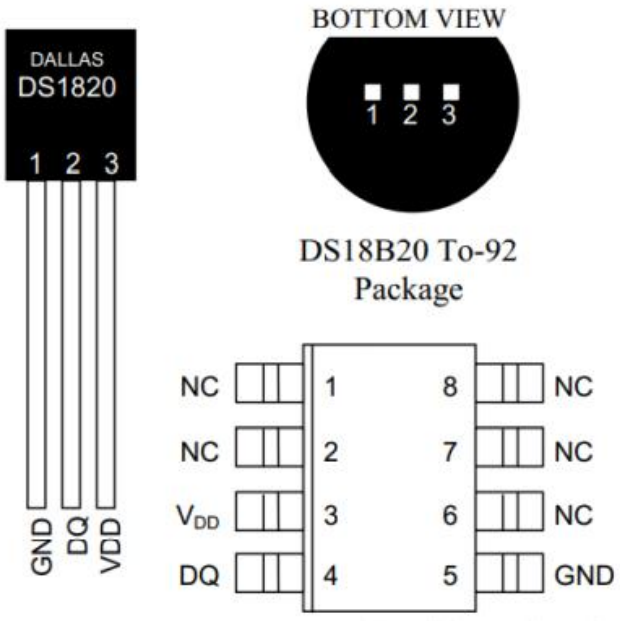
控制信号：图片中还包含了控制信号，用于协调整个音乐蜂鸣器系统的工作

```
always@(*)begin
case(note)
 'd01 :divnum='d190837; //freq =    'd262;
 'd02 :divnum='d170068; //freq =    'd294;
 'd03 :divnum='d151515; //freq =    'd330;
 'd04 :divnum='d143266; //freq =    'd349;
 'd05 :divnum='d127551; //freq =    'd392;
 'd06 : divnum='d113636;//freq =    'd440;
 'd07 :divnum='d101214; //freq =    'd494;

 'd11 : divnum='d095602;//freq =    'd523;
 'd12 :divnum='d085179; //freq =    'd587;
 'd13 :divnum='d075872; //freq =    'd659;
 'd14 :divnum='d071530; //freq =    'd699;
 'd15 :divnum='d063775; //freq =    'd784;
 'd16 :divnum='d056818; //freq =    'd880;
 'd17 :divnum='d050607; //freq =    'd988;
```

```
 assign pause=!note;
 always@(*)begin
case(num_reg)
'd0://qinghuaci
 case(s)
 'd0:note=0;
 'd1:note='d12;
 'd2:note='d11;
 'd3:note='d06;
 'd4:note='d11;
 'd5:note='d11;
 'd6:note='d11;
 'd7:note='d06;
 'd8:note='d11;
 'd9:note='d11;
 'd10:note='d11;
 'd11:note='d06;
 'd12:note='d11;
 'd13:note='d06;
```

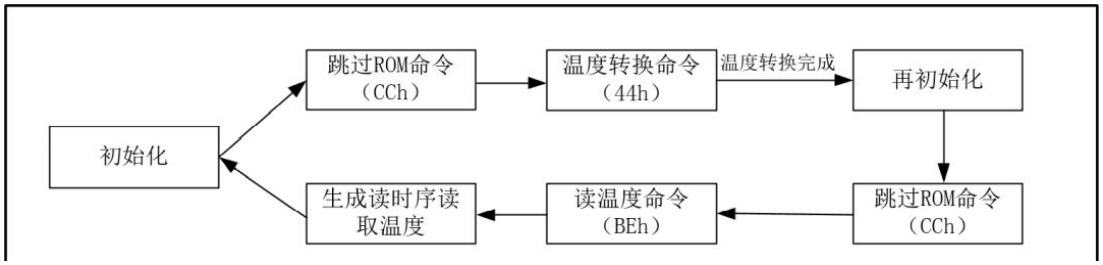
3.6 温度显示模块



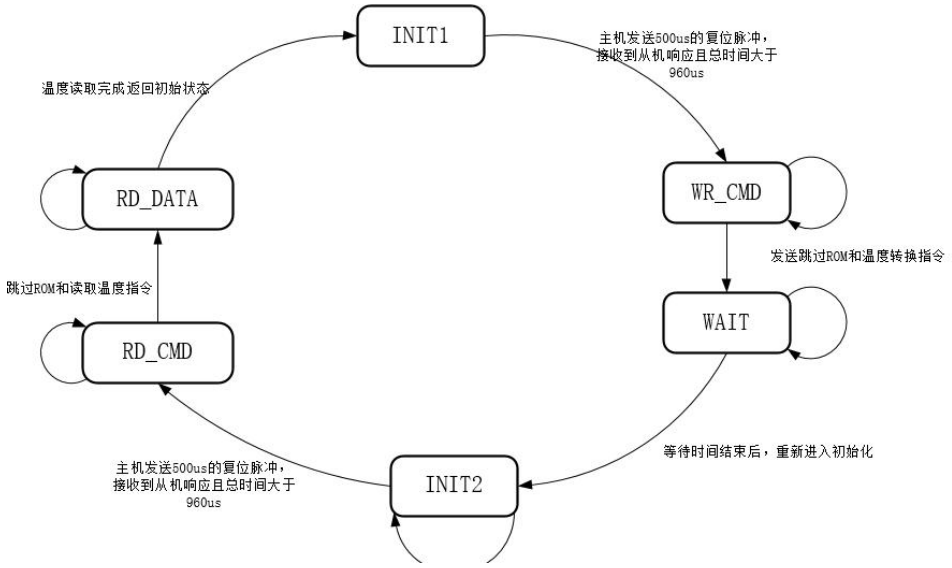
（DS1820 示意图）

温度	数据输出（二进制）	数据输出（十六进制）
+125°C	0000 0111 1101 0000	07D0h
+85°C	0000 0101 0101 0000	0550h
+25.0625°C	0000 0001 1001 0001	0191h
+10.125°C	0000 0000 1010 0010	00A2h
+0.5°C	0000 0000 0000 1000	0008h
0°C	0000 0000 0000 0000	0000h
-0.5°C	1111 1111 1111 1000	FFF8h
-10.125°C	1111 1111 0101 1110	FF5Eh
-25.0625°C	1111 1110 0110 1111	FF6Fh
-55°C	1111 1100 1001 0000	FC90h

（温度信号输出图）



（温度通讯步骤）



（状态机模式转换图）

具体描述了初始化（INIT）过程中主机与从机之间的交互步骤。以下是对描述中

步骤的解释：

1. NIT1（初始化步骤 1）：
   - 主机发送一个持续时间为 500 微秒（us）的复位脉冲给从机。
   - 从机接收到复位脉冲后，给出响应。
   - 从机响应与主机发送复位脉冲的总时间必须大于 960 微秒。
2. 温度读取：
   - 在完成初始化步骤 1 后，进行温度读取操作。
   - 读取完成后，系统返回到初始状态，等待下一次操作。
3. RD_DATA 和 WR_CMD（读取数据和写入命令）*：
   - 主机发送跳过 ROM（只读存储器）和启动温度转换的指令给从机。
   - 这是一个特定的命令，用于告诉从机忽略 ROM 中的信息，直接开始温度转换过程。
4. 等待时间（WAIT）：
   - 主机发送一个跳过 ROM 和读取温度的指令。
   - 从机在接收到指令后，需要等待一段时间来完成温度的读取或转换。
5. RD_CMD（读取命令）：
   - 等待时间结束后，从机需要发送一个读取命令，告知主机它已经准备好接收进一步的指令或请求。
6. INIT2（初始化步骤 2）：
   - 主机再次发送一个 500 微秒的复位脉冲。
   - 从机接收到复位脉冲后，给出响应。
   - 与步骤 1 相同，从机响应与主机发送复位脉冲的总时间必须大于 960 微秒。
7. 重新进入初始化：
   - 完成上述步骤后，系统重新进入初始化状态，准备进行下一轮的操作。

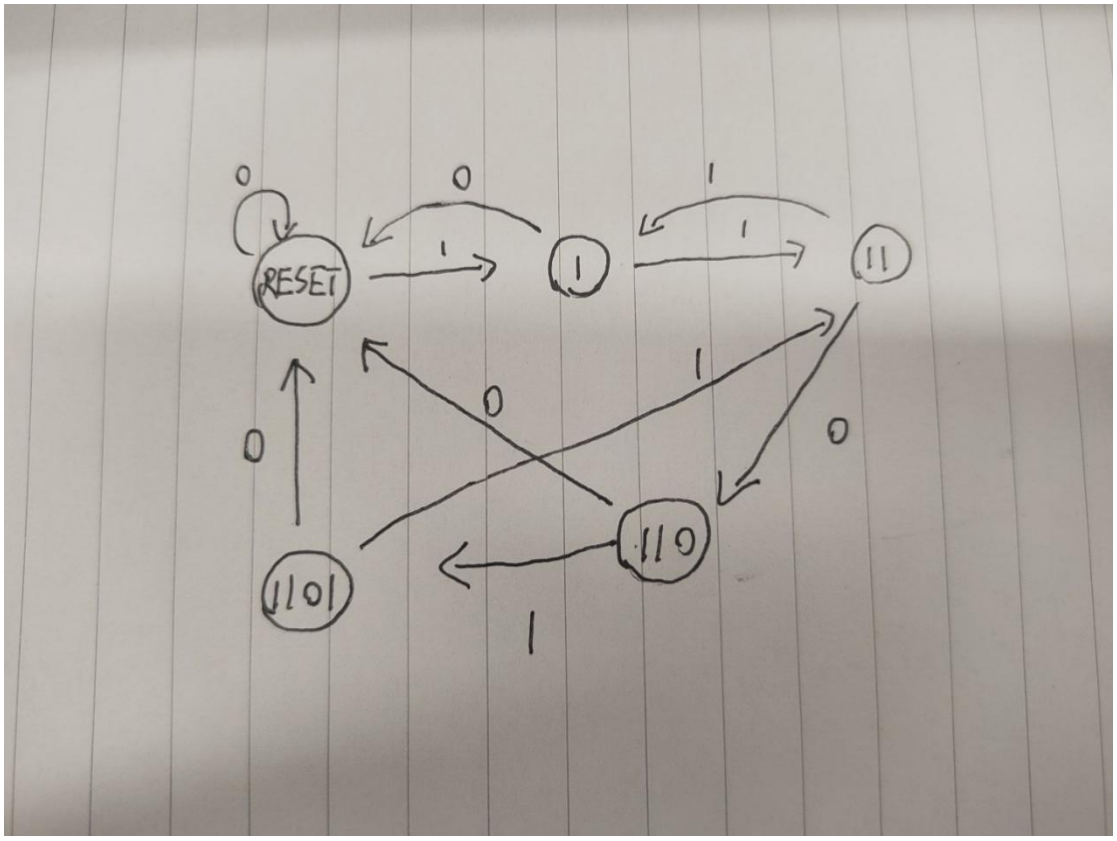
这些步骤描述了温度传感器与主机之间的通信协议，确保了设备正确地初始化并准备好执行其功能。每个步骤都涉及到特定的时间要求，以确保通信的可靠性和数据的准确性。

```
always @(*)begin
    next_state = INIT1;
    case(cur_state)
        INIT1       :begin
            if(cnt_us == T_INIT && flag_ack)
                next_state = WR_CMD;
            else
                next_state = INIT1;
        end
        WR_CMD      :begin
            if(bit_cnt == 4'd15 && cnt_us == 20'd62)
                next_state = WAIT;
            else
                next_state = WR_CMD;
        end
        WAIT  :begin
            if(cnt_us == T_WAIT)
                next_state = INIT2;
            else
                next_state = WAIT;
        end
        INIT2  :begin
            if(cnt_us == T_INIT && flag_ack)
                next_state = RD_CMD;
            else
                next_state = INIT2;
        end
        RD_CMD  :begin
            if(bit_cnt == 4'd15 && cnt_us == 20'd62)
                next_state = RD_DATA;
            else
                next_state = RD_CMD;
        end
        RD_DATA  :begin
            if(bit_cnt == 4'd15 && cnt_us == 20'd62)
                next_state = INIT1;
            else
                next_state = RD_DATA;
        end
        default:next_state = INIT1;
    endcase
```

（核心代码）

3.7 电子锁模块

仿照 LAB4 和 LAB5，基于有限状态机设计电子锁系统。系统通过状态机来控制锁的状态转换，并通过按键输入来触发状态转换。状态机的状态决定了七段显示器的显示内容。系统还包括了消抖和时钟分频等辅助功能，以确保稳定和可靠的操作。

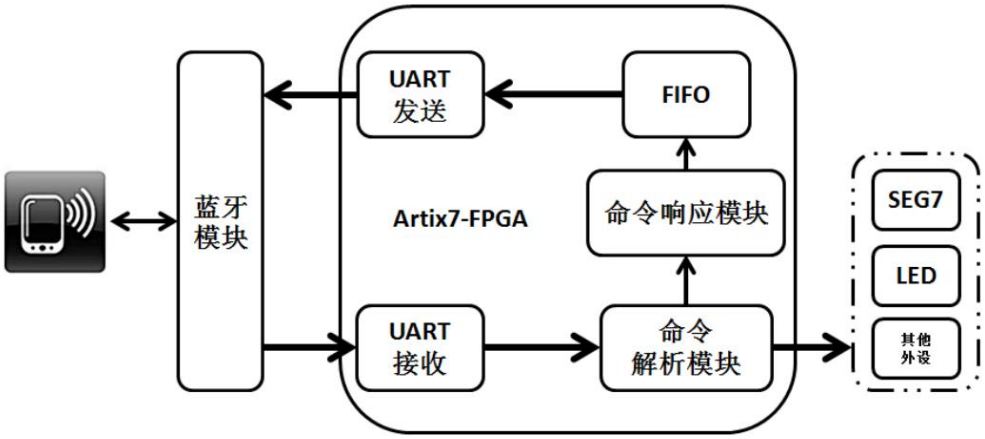


（电子锁模块状态转换图）

3.8 通讯模块

关于通讯模块，开题时定下的目标要比现在意义大的多，可以说我们组的整个设计都基于通讯模块能实现两版之间的无线传输。很遗憾最后也没能成功实现这一目的，最终不得已采用长线来连接，模拟远程控制的模式。

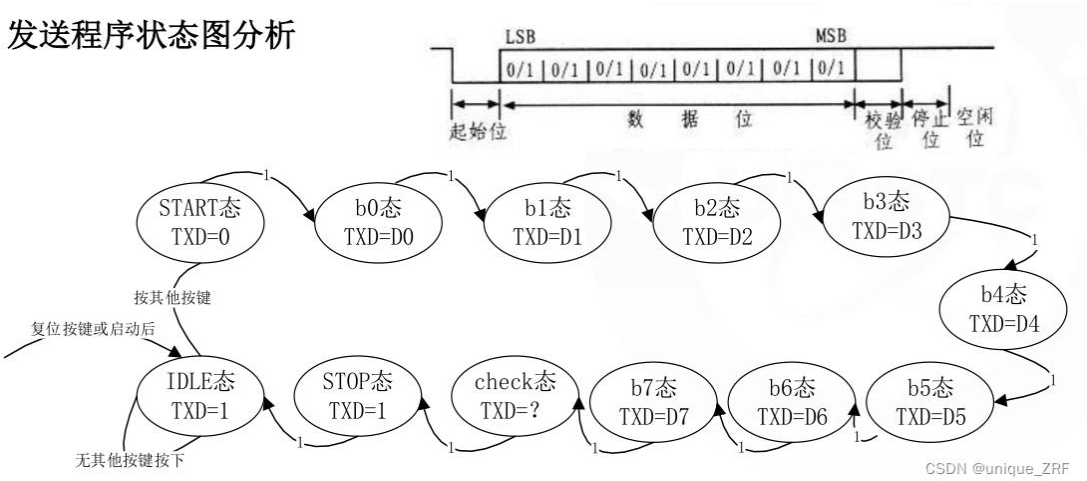
我们使用 Ego1 开发板上自带的蓝牙模块来实现蓝牙通讯，这就需要了解串口通信的相关协议。实现这一部分参考了开源的 EG01 开发板实验资料。



（蓝牙串口的实验框图）

本实验通过串口发送与串口接收模块来完成与蓝牙模块的数据传输。通过命令解析模 块及命令响应模块来实现简单的串口命令的解析控制以及命令的执行。FPGA 在接收到蓝牙 模块传输进来的串口数据后，会将相应数据以及命令响应通过蓝牙模块发送给与之连接的通 信设备，在这个过程中采用 FIFO 来存储我们需要发送的数据。

发送程序状态图分析

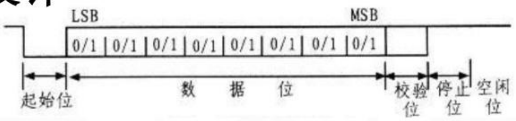


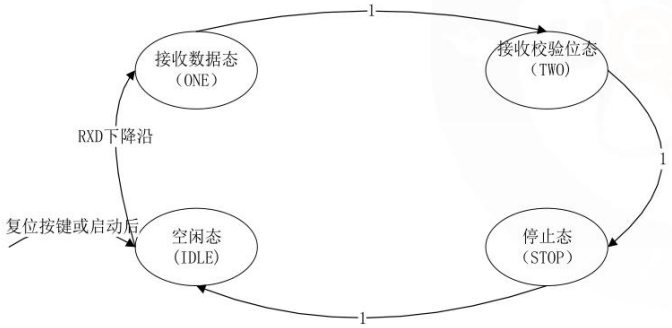
收发的过程，在发送器空闲时间，数据线处于逻辑 1 状态，当提示有数据要传输时，首先使数据线的逻辑状态为低，之后是 8 个数据位、一位校验位、一位停止位，校验一般是奇偶校验，停止位用于标示一帧的结束，接收过程亦类似，当检测到数据线变低时，开始对数据线以约定的频率抽样，完成接收过程。数据帧采用：无校验位，停止位为一位。

系统启动后进入空闲态 IDLE，如果按了实验电路板中间的按键（S2）也可以进入 IDLE 状态。如果有其他的按键按下，开始发送数据，数据为拨码开关设置的 8 位数值，在时钟的有效边沿进入 START 状态发送起始位，时钟选用 clk_x 约为 9600Hz。

串行接收程序设计

接收程序状态图





CSDN @unique_ZRF

由上图可知，将串行接收状态划分为 4 个主要状态。系统启动后进入空闲态 IDLE，如果捕捉到了 RXD 信号的下降沿，开始接收数据启动计数。因为这里是启动信号 0 的开始，所以延后 24 个时钟，24=16+8，就大约到了数据位 0 的中间，在这个时间点上才转换到接收数据态 ONE。

在接收数据态 ONE，每隔 16 个时钟，就采集一次数据，分别存放到寄存器 data_out 的对应位。在第 136 分时钟开始之后的 16 个时钟，处于检查校验的状态 TWO，判断校验是否有误，如果有误，则置位校验出错信号 data_error，如果无误，则置位数据准备好信号 data_ready。之后进入 STOP 状态，等待时钟计数到 168，清时钟计数回到 IDLE 态，继续监测串行输入 RXD 信号的到来。

由于对 AT 指令和两蓝牙之间握手协议的不了解，我们最终未能实现板间通讯，只能退而求其次选择了手机与板之间互发信号，只能说基本上实现了上述的设计，但还有很大的不足。

四．心得体会

4.1 郑智瑄：

一学期匆匆而过，刚开始上这门课的我应该没想到能做出这么一个看起来还蛮“高科技”的东西的吧。从简单的 ALU 实现，到一个状态机密码锁，再到一个专用微处理器，最后到一个通用微处理器，本以为这些 lab 的难度跨越已然很大，但最后的大作业真的是难度陡升。着手的时候以为传感器的实现很简单，没想到被通讯 IIC 卡了很久很久，最终还是在老师和助教的帮助下勉强通过，相比之下后面的进制转换，阈值处理反而变得简单。感觉这一块最有“灵气”的地方还是在发现“晃动”不稳定后改用“加速度贯穿”法来实现，看似一个简单的替换其实对整个系统的稳定性，呈现效果有很明显的提升。然后是过渡模块，这个模块是一个小的专用微处理器，看似在系统中没有很明显的起作用，但却的的确确的增加了整个系统的流畅度，而且是对 lab 学习后的一个较好运用。VGA 我感觉是一个有无限潜能的创作平台，这次仅仅只是实现了显示，动画，体积碰撞三个内容，其实还有很长的路可以走。但使用 FPGA 决定了从刚开始的驱动就需要自己编写，对着时序图设计合规的状态机反而会更难。感觉作为一个不太深入的课程作业，对于我这样的小白来说，FPGA 设计更难的地方反而在于驱动，还有整体层面的规划，中间的代码层并不会特别的复杂。不管怎样，这次的课程设计确实让我收益良多，为我打开了专业的大门。感谢老师与助教，感谢组员们!

4.2 李明远：

本次课程设计我们小组以手套遥控为主题，我负责的是温度读取和音乐播放两个控制对象。首先是音乐播放器模快，这部分相对简单，通过状态机控制输出信号的频率即可输出音频信号，主要工作为使用分频器把时钟频率转换为音调频率，以及构建循序播放音符的状态机。在成功播放一段音乐后，经过小组讨论，我又为音乐播放器添加了上一首、下一首、暂停等功能 。关于温度传感器模块，我使用的 DS18B20 数字传感器采用单总线传输，所以通信协议是重中之重。了解了该传感器的内部结构和处理数据的时序后，我和其他组组员和老师共同交流，

设计出了对应的状态机以读取数据。

此外，通过此次课程设计我充分认识到，一个优秀的数字系统设计绝非局限于数字系统相关领域知识，更是与其他领域的结合。例如，在设计音乐播放器前，我学习了简谱中不同音调的种类，拍数和音符演奏时长的关系等等。而且，我们小组使用温度传感器的初衷是通过温控实现不同设备的切换，但当我们最终将其做成并使用后才发现，温度传感器响应速度慢，降温速度慢，很不灵敏，所以我们最终舍弃温控采用键控，而温度传感器则作为温度显示模块使用。在答辩后我们小组反思，光控是一个极好的方案，不仅响应迅速，而且极为精准。所以我们小组在动手去做前应对此进行详细调研，以选出最优方案。

进行了一学期的数字系统学习，我充分感受到这门课程把理论、编程和实践紧密地联系在一起。通过此次课也正式打开了我通向信电的大门，与队友、各位同学和老师们的交流也让我受益良多。

4.3 赵欧凯：

整门课学下来感觉数字系统是门探索能力极强的课程，后面的几个 lab 都顺利完成，让我在开始动手写大作业之前感到信心十足，甚至开题时还有多使用几种不同的无线通讯方式来完成通讯模块的设计，可很快现实就给了我一次次的打击，也一次次的刺激了我的学习能力。

刚开始设计时，我发现要实现板子将信号传输出去，需要采用串口通讯的方式，而板子上所带有的 uart 部分仅有两个，一个 USB 接口一个蓝牙。于是我将目光放在了 USB 接口上，令人沮丧的是，使用的无论是 RS485 无线通信模块或是 ESP8266 WIFI 芯片在接上 USB 串口后，都无法正常激活使用，所以我自然的将希望放在了板子自带的蓝牙串口上，查阅了一些资料后我感觉利用该模块实现通信功能应该没有太大问题，谁曾想后面都困在这个圆圈里。

正所谓：“正入万山圈子里，一山放过一山拦”。将设计转向蓝牙模块后也是困难重重，后来明白还是知识储备不够。开始时，我认为蓝牙既然是串口通信，那么控制串口的代码自然可以控制蓝牙，所以参考了几个串口通信的例子，完成了初步的代码，这时我发现代码跑到板子上之后，手机搜不到蓝牙信号。我以为板

子之间有特定的频段，但尝试采用两板连接之后，发现不是没有建立连接，是根本没有激活蓝牙，这令我百思不得其解。中期和李老师交流时，老师建议我先建立蓝牙和手机的连接，一步步来，于是我放弃了原有的想法，求助了网上开源的相关资料。经过不断的尝试和试错，实现了手机命令发给板子执行，板子返回信号给手机的功能。当时以为离成功已经很近了，可后面进行两版互联时仍无法成功，这时班里做这部分的同学大多遇到了同样的问题，于是我们求助了余老师，了解了蓝牙之间需要有主从关系，才能建立握手。在尝试解决这部分后，我们发现后面还有 AT 命令在等着我们去解决……

一个一个的问题不断的浮现，使得我们最终也没能实现既定的目的。但探索过程是无比有意义的，这段经历让我体会到了团队合作的快乐和动力，来自不同的小组做不同的项目的同学之间通力合作，共同解决面临的相似问题，古语云，学学半，虽然最终没能达成目标，也让我们所有人都得到了成长。我很遗憾到此时才发现这个道理。

五．成员分工

5.1 进度介绍：

夏四周：构建加速度模块，音乐播放器模块，通讯模块设计思路；进行初步设计

夏五周：完成基础加速度模块，过渡模块和音乐播放器模块设计，手机实现与开发板蓝牙连接，尝试进行级联

夏六周：与老师沟通进度，设计 VGA 模块和密码锁模块；尝试两块板子通讯连接未成功

夏七周：完成 VGA 模块，密码锁模块初始设计，开始设计温度显示；同时周末进行第二次级联测试。蓝牙通讯在与班级同学交流后了解到控制办法，仍不能实现两板之间通讯。

夏八周：完成温控模块，放弃蓝牙通讯，将之前的设计简化为利用手机蓝牙点灯；进行最后的级联测试，拍实验视频，完成答辩 ppt 和实验报告。装置维护与细节优化 ，答辩。

5.2 分工与贡献比;

郑智瑄：负责加速度传感模块，过渡模块，VGA 模块，密码锁优化，报告中文稿编写与图表制作。贡献占比：35.5%

李明远：负责音乐播放器模块，温控模块，报告中亦负责文稿编写与图表制作。贡献占比：34.5%

赵欧凯：负责通讯模块 ，密码锁初始设计，报告中负责图表优

化与 PPT 制作 。贡献占比：30%

六、物资清单与价格

物品清单	金额/元
纯铜加粗杜邦线	46.31
户外学生骑行手套	39.9
VGA 连接线	9.8
一字微型螺丝批	11.48
ADXL345 加速度传感器×3	28.02
DS18B20 温度传感器开发板	4.9
有线音响	20.9
共计	**161.31**